Educação Profissional Paulista

Técnico em **Desenvolvimento de Sistemas**

Secretaria da Educação

SÃO PAULO
GOVERNO DO ESTADO

Introdução ao versionamento de código, Git e GitHub

Colaboração e fluxo de trabalho com o GitHub

Aula 3: Comentários, issues e histórico

Código da aula: SISANO2C5B1S4A3

Secretaria da Educação
SÃO PAULO GOVERNO DO ESTADO

Mapa da Unidade 1 Componente 5

Introdução ao versionamento de código, Git e GitHub

semana 1

Fundamentos do versionamento de código

semana 2

Conhecendo as ferramentas existentes no mercado

semana 3

Introdução ao GitHub

Você está aqui!

semana 4

Colaboração e fluxo de trabalho com o GitHub

Introdução ao versionamento de código, Git e GitHub

Mapa da Unidade 1 Componente 5

Você está aqui!

Colaboração e fluxo de trabalho com o GitHub

4

Aula 3: Comentários, issues e histórico

Código da aula: SISANO2C5B1S4A3

Objetivos da aula

- Utilizar ferramentas de comunicação e rastreabilidade no GitHub.

Recursos didáticos

- Roteiro de Atividade Prática.
- Recurso audiovisual para exibição de vídeos e imagens.
- Caneta e caderno para anotações.
- Computador com internet.

Duração da aula

50 minutos.

Habilidades técnicas

- Compartilhar repositórios, colaborar com outros usuários e acompanhar alterações em equipe.

Habilidades socioemocionais

- Valorizar a colaboração e a troca de experiências, demonstrando interesse em aprender a dividir tarefas com responsabilidade e segurança.

Colocando em **prática**

Comunicação profissional no GitHub – Issues, comentários e histórico

Materiais necessários

- Computador com ambiente de desenvolvimento;
- Editor de texto ou IDE para pseudocódigo;
- Caderno para rascunhos;
- Caneta.

Passo a passo

- Leiam com atenção a atividade proposta no roteiro.
- Antes de iniciar a codificação, façam esboços no papel para definir a lógica.
- Utilizem as ferramentas de desenvolvimento disponíveis para implementar a atividade.

45 minutos

Em grupos

Baixe o roteiro dessa atividade

Pause e **responda**

Registro

Qual é a melhor forma de reportar um bug encontrado em um projeto no GitHub?

- Enviar e-mail para o desenvolvedor.
- Criar uma issue detalhada.
- Comentar no último commit.
- Editar o README com o erro.

Pause e **responda**

Registro

Qual é a melhor forma de reportar um bug encontrado em um projeto no GitHub?

- ✗ Enviar e-mail para o desenvolvedor.
- ✓ Criar uma issue detalhada.
- ✗ Comentar no último commit.
- ✗ Editar o README com o erro.

Então ficamos assim...

1. Compreendemos que issues são a forma profissional de reportar bugs e sugerir melhorias.

2. Entendemos que comentários em commits e PRs mantêm discussões contextualizadas e histórico centralizado.

3. Descobrimos que a comunicação clara no GitHub evita mal-entendidos e retrabalho.

Referências da aula

ALURA. **O que é Git e GitHub: os primeiros passos nessas ferramentas**, 16 nov. 2020. Disponível em: https://www.alura.com.br/artigos/o-que-e-git-github. Acesso em: 8 set. 2025.

CÓDIGO FONTE TV. Git: minicurso para você sair do zero! (Aprenda em 45 minutos). Disponível em: https://www.youtube.com/watch?v=ts-H3W1uLMM. Acesso em: 8 set. 2025.

FERNANDES, D. Git & GitHub: o que é? Por quê? Como iniciar? **Rocketseat**, [s.d.]. Disponível em: https://www.rocketseat.com.br/blog/artigos/post/git-e-github-o-que-e-por-que-como-iniciar. Acesso em: 8 set. 2025.

Identidade visual: imagens © Getty Images

Orientações ao professor

Slide 6

Orientações: a atividade prática tem como objetivo:
Fortalecer a comunicação técnica via issues, prs e revisões no seu repositório;
Praticar organização de trabalho com papéis rotativos (reporter, developer, reviewer);
Exercitar resolução de conflitos, vínculo pr↔issue e registro de decisões no histórico;
Refletir sobre governança (permissões, templates, branch protection) e métricas de colaboração.

Tempo previsto: 45 minutos.

Gestão de sala de aula:
1. Preparação do professor (antes da aula):
Garantir que todos tenham repositório próprio e acesso ao github;
Exibir/compartilhar um quadro de comandos mínimos: git status, git add ., git commit -m, git pull --rebase origin main, git push, git log --oneline --graph;
Lembrar regras de segurança: nada de segredos no repo; usar .Gitignore;
Ter um slide/modelo de issue (bug/feature/question) e um PR template para referência.

2. Organização da turma:
Trios com papéis rotativos:
Pessoa A (reporter): abre 3 issues profissionais no repositório escolhido do trio;
Pessoa B (developer): responde, rotula, cria branch, abre PR e referencia issue;
Pessoa C (reviewer): revisa PR (comentários em linha), sugere ajustes, aprova.
Alterne papéis após concluírem um ciclo issue→pr→merge (se houver tempo, rode 2 ciclos, trocando papéis).

.

Continua...

Slide 6

Condução da dinâmica:

Alinhamento do objetivo
Professor(a), relembre: o foco é comunicação e rastreabilidade (issues bem escritas, PRs com contexto, revisão respeitosa).

Setup do trio
Owner convida colaboradores; verificam Manage Access. Confirmam que issues estão ativas.

Ciclo 1 – Do problema ao merge

A cria 3 issues (bug, feature, question) com labels e critérios de aceite.

B responde, prioriza, cria branch (fix/… ou feat/…), faz commit, abre PR e referencia ("Fixes #id").

C revisa em "Files changed" (comentários objetivos), solicita ajustes se necessário e aprova.

B faz merge (sugira squash) e fecha a issue (automático pelo "Fixes").

A valida a correção/feature e comenta o resultado na issue.

Conflito controlado
Gerar um pequeno conflito (mesma linha no README.md) e resolver com git pull --rebase origin main, marcadores <<<<<<<</=======/>>>>>>>, commit de resolução e push.

Explorar histórico e métricas do próprio repo.

git log --oneline --graph (antes/depois do conflito).

Insights → Pulse/Contributors/Network, Blame em um arquivo.

Reflexão final
Cada trio registra respostas das 3 perguntas dissertativas (as que já definimos) no repositório (ex.: docs/reflexao-colaboracao.md) e referencia issues/PRs.

Continua...

Slide 6

Expectativas de respostas:

1) descrevam, passo a passo, como uma issue aberta pelo reporter foi priorizada, tratada pelo developer e revisada pelo reviewer até o merge. Expliquem as decisões tomadas (rótulos/labels, milestone, critérios de aceite); como o PR foi vinculado à issue (**fixes/closes #id**); os comentários em linha que mudaram o código; e o que aprenderam com esse ciclo.

O que esperamos ver:
Clareza de priorização (por que essa issue primeiro);
Uso de labels/milestone/critérios de aceite;
PR vinculado à issue; comentários em files changed influenciando o código;
Evidências: links/ids de issue e PR, resumo do diff e resultado do merge;
Reflexão sobre comunicação técnica (tom, objetividade, rastreabilidade).

2) relatem um conflito real (ou simulado) que ocorreu no repositório: como foi detectado, quais arquivos/linhas estavam envolvidos e qual estratégia usaram para resolver (pull --rebase, merge, squash). Comparem prós e contras das abordagens escolhidas para clareza do histórico**, rastreabilidade e colaboração.**

O que esperamos ver:
Descrição do conflito (marcadores <<<<<<<, =======, >>>>>>>) e resolução;
Justificativa de rebase vs. Merge vs. Squash no contexto do time;
Evidências: trecho do histórico (git log --oneline --graph) antes/depois;
Boas práticas: mensagens de commit, commits pequenos, evitar arquivos indevidos, .Gitignore.

3) analisem as políticas de colaboração que adotaram (permissões write, revisão obrigatória, templates de issues/pr, branch protection se aplicável).
Proponham melhorias para quando houver mais pessoas contribuindo. Definam 3 métricas para acompanhar a saúde do fluxo (ex.: Lead time for changes, tempo médio de PR aberto, taxa de merge sem revisão, taxa de conflitos) e expliquem como mediriam cada uma.

O que esperamos ver:
Avaliação crítica de permissões e do processo de revisão;
Sugestões concretas: CODEOWNERS, status checks, squash merge, PR template;
Três métricas bem definidas (o que é, como coletar, meta desejada);
Relação entre práticas e redução de risco (menos conflitos, mais transparência).

Slides 7 e 8

Orientações: professor(a), use para verificar a compreensão básica do conceito. Explique por que cada alternativa está correta ou incorreta. Conecte com exemplos do cotidiano.

Tempo previsto: 5 minutos.

Alternativa correta: Criar uma issue detalhada.

Feedback: Isso garante que o problema seja rastreável e permite discussões entre os envolvidos, a notificação dos interessados e a permanência no histórico do projeto, seguindo o padrão da indústria.

Educação Profissional Paulista

Técnico em
Desenvolvimento de Sistemas

Secretaria da Educação

SÃO PAULO
GOVERNO DO ESTADO